Universidade Presbiteriana Mackenzie
Mestrado Profissional em Computação Aplicada

Professora: Valéria Farinazzo Martins

Disciplina: Algoritmos e Programação

Aluno: Cândido Francisco Moreira

RA: 10779363

Período: 1º Semestre de 2026

Projeto 1 - Métodos de Ordenação

Relatório Métodos de Ordenação

São Paulo, 08 de março de 2026

1. Resumo / Introdução

A ordenação de dados constitui um dos problemas fundamentais da ciência da computação, sendo amplamente utilizada como etapa intermediária em diversos processos computacionais, como busca, indexação, análise estatística e manipulação de grandes volumes de informação. Ao longo do desenvolvimento da área, múltiplos algoritmos de ordenação foram propostos, cada um apresentando características distintas em termos de complexidade temporal, consumo de memória e comportamento diante de diferentes distribuições de dados. Nesse contexto, compreender não apenas a análise assintótica desses algoritmos, mas também seu comportamento empírico em ambientes reais de execução, torna-se relevante para a escolha de estratégias eficientes em aplicações práticas.

O presente trabalho apresenta um estudo comparativo abrangente sobre a eficiência empírica e a complexidade assintótica de algoritmos clássicos de ordenação baseados em comparação. A pesquisa engloba a implementação e a avaliação estrutural do Bubble Sort (em suas variantes padrão e otimizada), Selection Sort, Insertion Sort, Merge Sort e Quick Sort sob um ambiente de execução interpretado. Para aferir o comportamento temporal e a resiliência dessas abordagens, o delineamento experimental submeteu os algoritmos a diferentes magnitudes de carga — variando até instâncias massivas de 100.000 elementos — e a três cenários distintos de permutação inicial: vetores uniformemente aleatórios, estritamente crescentes e estritamente decrescentes. Através de uma metodologia pautada no rigor estatístico para a mitigação de ruídos de hardware e do sistema operacional, o objetivo central deste ensaio é confrontar as premissas teóricas da notação Big-O com o desempenho prático computacional. Dessa forma, busca-se fornecer uma fundamentação crítica e arquitetural sobre a escalabilidade, o custo computacional e a estabilidade inerentes a cada paradigma algorítmico, independentemente das abstrações teóricas.

2. Metodologia Experimental

Os experimentos empíricos foram delineados e conduzidos com o rigor necessário para avaliar o comportamento temporal e a eficiência arquitetural de diferentes algoritmos clássicos de ordenação baseados em comparação, confrontando seus desempenhos práticos sob variados cenários de estresse computacional.

1.1 Ambiente Computacional e Instrumentação

Os testes foram executados em um sistema controlado com as seguintes especificações de *hardware* e *software*:

- **Processador:** Intel Core i7-14700KF (20 núcleos híbridos, arquitetura x86-64).
- **Memória RAM:** 32 GB DDR5 (capacidade do sistema 64GB). Essa capacidade foi estritamente dimensionada para acomodar as alocações massivas no *heap* da memória principal, garantindo que o volume de dados não introduzisse efeitos de paginação (*swap*) no disco. Isso permitiu que a análise ficasse isolada e concentrada predominantemente na complexidade algorítmica.
- **Ambiente de Execução:** Interpretador Python 3.12.4 (CPython, 64-bits).
- **Sistema Operacional:** Windows 11 Pro (64-bits)
- **Aferição de Tempo:** As medições foram capturadas utilizando a função nativa time.perf_counter, que fornece um relógio de alta resolução e contínuo, imune a ajustes arbitrários de tempo do sistema, garantindo alta fidelidade na cronometragem de frações de segundo.

1.2 Procedimento Experimental e Rigor Estatístico

Para mitigar interferências espúrias provenientes do escalonamento do sistema operacional — tais como processamentos em segundo plano, varreduras de antivírus ou variações momentâneas de carga na CPU — a coleta de dados não se baseou em execuções singulares.

Cada ensaio foi executado k=5 vezes de forma independente para cada combinação de algoritmo, tamanho de vetor e permutação inicial. A partir desse conjunto amostral, foram extraídas as seguintes métricas estatísticas:

- **Média Aritmética:** Para determinar o tempo de execução representativo do cenário.
- **Desvio Padrão:** Para estimar a variabilidade do desempenho e quantificar o impacto de ruídos experimentais transientes, garantindo a confiabilidade dos dados obtidos.

1.3 Espaço de Busca e Instâncias de Teste

O espaço amostral foi submetido a três magnitudes vetoriais de carga: {1.000}, {10.000} e {100.000} elementos. Para cada dimensão, foram gerados três padrões topológicos de entrada:

- **Distribuição Aleatória:** Elementos dispostos de forma estocástica.
- **Estritamente Crescente:** Vetor previamente ordenado em sua totalidade.
- **Estritamente Decrescente:** Vetor com arranjo estrutural completamente invertido.

A modelagem paramétrica desses três cenários foi definida intencionalmente para forçar os algoritmos e permitir a observação empírica dos comportamentos teóricos associados ao **melhor caso**, **caso médio** e **pior caso** de execução.

1.4 Algoritmos Avaliados

A bateria de testes contemplou a implementação e a submissão dos seguintes algoritmos à suíte de avaliação:

- Bubble Sort (versão padrão)
- Bubble Sort (versão otimizada com interrupção prematura)
- Selection Sort

- Insertion Sort
- Merge Sort
- Quick Sort (com seleção de pivô central para mitigação de pior caso)

Para garantir a transparência e a total reprodutibilidade computacional dos experimentos realizados, todos os códigos-fonte — incluindo as implementações dos algoritmos de ordenação, os *scripts* de geração de instâncias de dados e as rotinas de execução e aferição estatística — foram disponibilizados publicamente no repositório do autor no GitHub[1].[1]

3. Resultados Experimentais

A Tabela 1 apresenta as médias de tempo de execução observadas para vetores de **100.000 elementos**, representando o cenário de maior carga computacional testado.

Algoritmo	Média (Aleatório)	Média (Crescente)	Média (Decrescente)
Bubble Sort Padrão	~241 s	~147 s	~338 s
Bubble Sort Melhorado	~261 s	~0.00 s	~358 s
Selection Sort	~109 s	~107 s	~123 s
Insertion Sort	~110 s	~0.00 s	~234 s
Merge Sort	~0.15 s	~0.11 s	~0.12 s
Quick Sort	~0.09 s	~0.06 s	~0.07 s

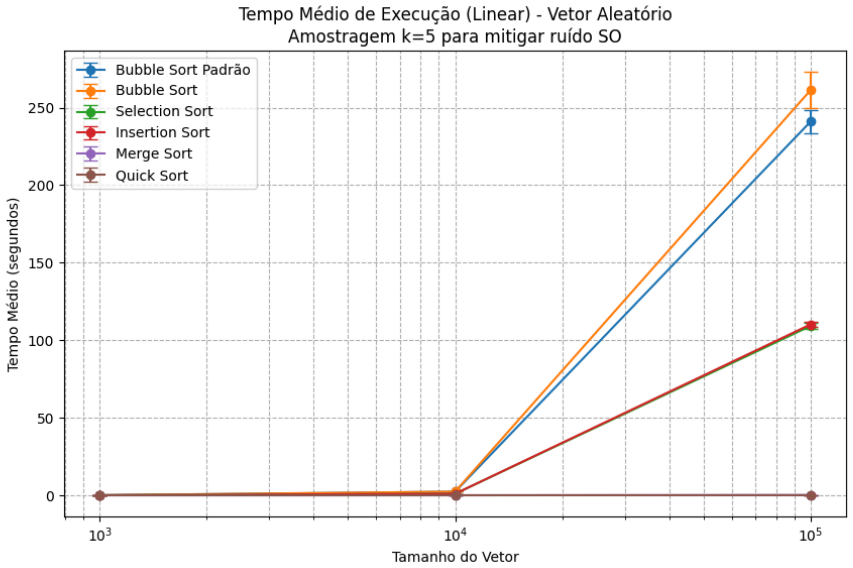
Os gráficos gerados ao longo dos experimentos evidenciam uma separação clara entre duas classes de algoritmos:

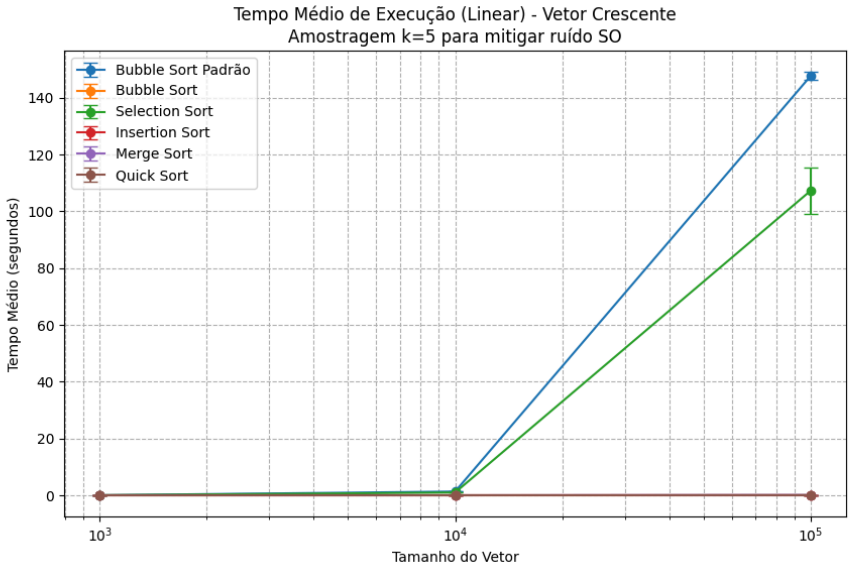
Algoritmos quadráticos: Bubble, Selection e Insertion

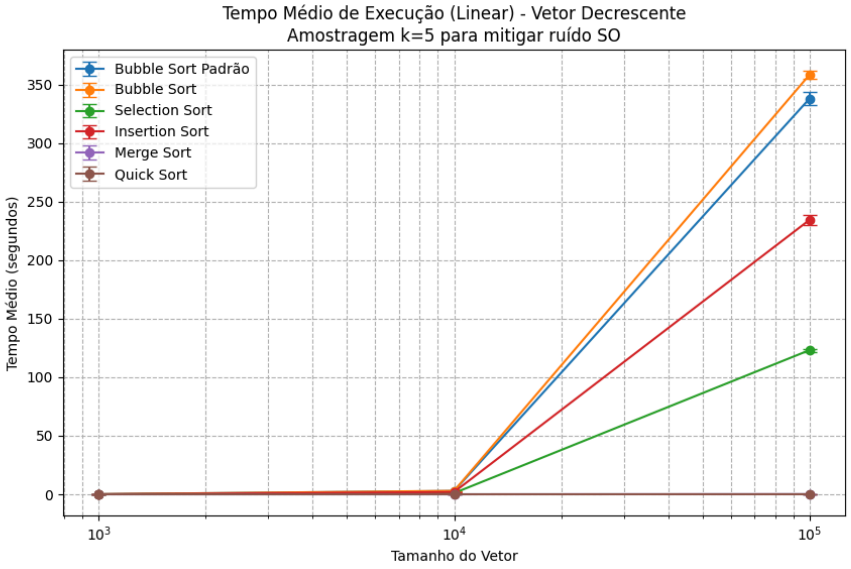
Algoritmos log-lineares: Merge e Quick Sort

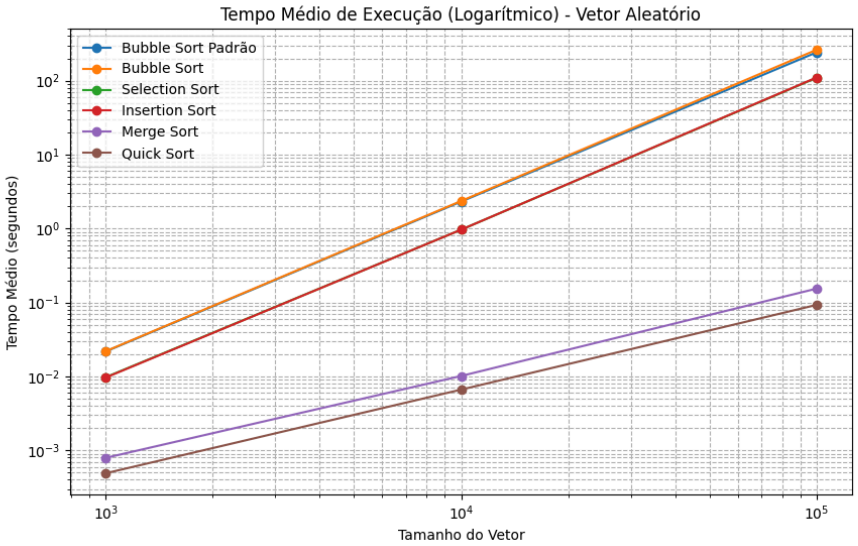
A diferença de desempenho entre essas classes torna-se particularmente evidente conforme o tamanho da entrada cresce.

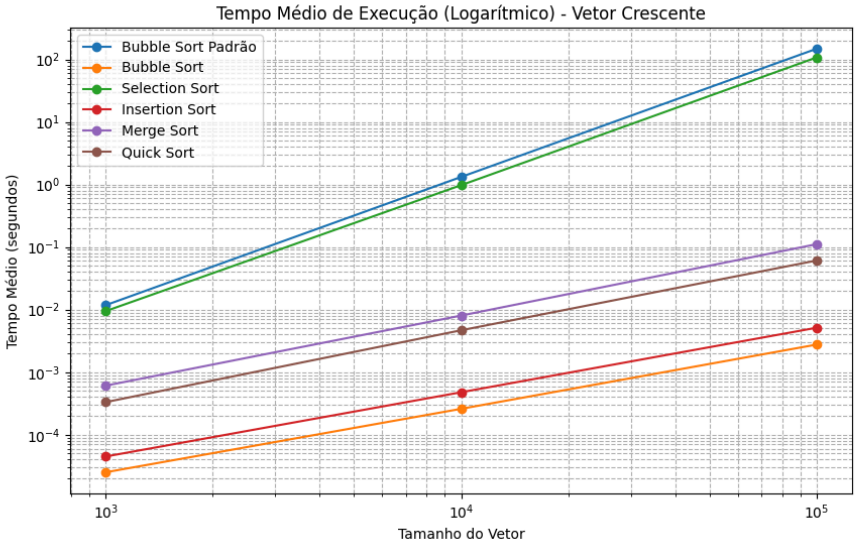
[1] Repositório disponível em: https://github.com/candidomor/Mestrado-Algoritmos-Projeto1/tree/main/Principal

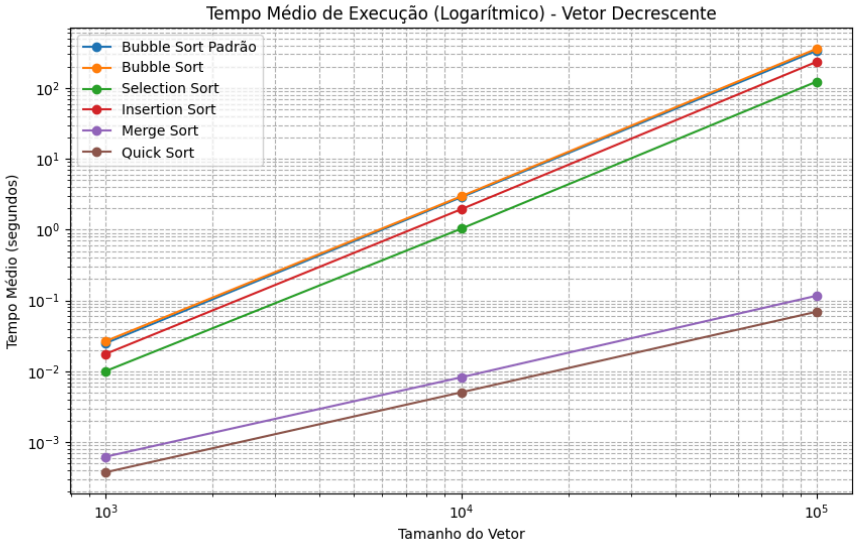

Tempo Médio de Execução (Linear) - Vetor Aleatório
Amostragem k=5 para mitigar ruído SO
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor


Tempo Médio de Execução (Linear) - Vetor Crescente
Amostragem k=5 para mitigar ruído SO
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor


Tempo Médio de Execução (Linear) - Vetor Decrescente
Amostragem k=5 para mitigar ruído SO
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor


Tempo Médio de Execução (Logarítmico) - Vetor Aleatório
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor


Tempo Médio de Execução (Logarítmico) - Vetor Crescente
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor


Tempo Médio de Execução (Logarítmico) - Vetor Decrescente
Bubble Sort Padrão
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Tempo Médio (segundos)
Tamanho do Vetor

4. Análise Comparativa da Eficiência dos Algoritmos

Os resultados obtidos demonstram empiricamente o impacto substancial da complexidade assintótica na escalabilidade dos algoritmos. Nos experimentos de estresse submetidos à carga de 100.000 elementos, evidenciou-se um contraste computacional extremo: enquanto os métodos baseados em comparações elementares apresentaram tempos de execução na ordem de centenas de segundos, as abordagens fundamentadas no paradigma de Divisão e Conquista completaram a mesma tarefa em frações decimais de segundo.

Esse comportamento ilustra e materializa a drástica diferença de desempenho prático entre as classes de complexidade **$O(n^2)$** e **$O(n \log n)$** quando aplicadas a grandes volumes de dados.

4.1 Quick Sort e a Eficiência *In-Place*

O Quick Sort apresentou o menor tempo médio de execução absoluto entre todos os algoritmos avaliados. Esse desempenho superior deve-se, primordialmente, à sua mecânica de operar *in-place* (realizando as permutações de elementos no próprio arranjo). Essa característica minimiza a necessidade de alocações dinâmicas no *heap* da memória e maximiza o aproveitamento da localidade de referência na memória cache do processador. O contraponto arquitetural dessa eficiência bruta é que a implementação tradicional do algoritmo é **instável**; isto é, ele não oferece garantias matemáticas de que a ordem relativa original de chaves idênticas será preservada após a ordenação.

4.2 Merge Sort: Estabilidade e Determinismo

O Merge Sort exibiu um desempenho temporal levemente inferior ao do Quick Sort. Essa discreta penalidade de tempo é diretamente atribuída ao custo espacial auxiliar inerente ao seu processo de fusão (*merge*), que exige a alocação de memória extra na ordem de **$O(n)$** para a criação de subvetores.

Em contrapartida, o algoritmo compensa esse *overhead* de memória oferecendo duas propriedades estruturais de altíssimo valor:

- **Estabilidade:** Garante rigorosamente a preservação da ordem espacial ou cronológica de registros equivalentes (chaves com o mesmo valor).
- **Determinismo Assintótico:** Mantém uma complexidade de tempo garantida e inabalável de **$O(n \log n)$**, independentemente da distribuição topológica inicial dos dados (seja no melhor, médio ou pior caso experimental).

Essas características tornam o Merge Sort a escolha metodológica ideal dentre as opções testadas e mais segura para sistemas que manipulam estruturas de dados complexas, onde o histórico subjacente dos registros importa e os atributos secundários não podem ser embaralhados durante o processo de classificação.

5. Relação entre Análise Assintótica e Resultados Empíricos

Os dados extraídos da experimentação demonstram empiricamente o impacto da complexidade assintótica na escalabilidade dos algoritmos, ilustrando com clareza o reflexo das abstrações matemáticas na prática computacional.

5.1 Bubble Sort: Comparação entre Versão Padrão e Otimizada

A versão otimizada do Bubble Sort introduz um mecanismo de interrupção antecipada, acionado pela detecção de ausência de trocas (*swaps*) ao longo de uma iteração completa. Nos ensaios com instâncias estritamente crescentes, essa otimização reduziu o tempo de execução para valores limítrofes a zero (0,00 s), confirmando de forma irrefutável o melhor caso linear da notação **$\Omega(n)$**. Em contrapartida, o Bubble Sort padrão, estruturalmente engessado na complexidade **$\Theta(n^2)$**, exauriu aproximadamente 147 segundos sobre a mesma matriz já ordenada.

Curiosamente, nos cenários aleatórios e decrescentes, a versão otimizada apresentou um desempenho temporal ligeiramente inferior à sua contraparte padrão. Esse fenômeno comprova que a atribuição booleana e a verificação condicional de controle de fluxo introduzem uma penalidade de constante intrínseca no interpretador do CPython, um custo que se torna perceptível e punitivo quando o algoritmo inevitavelmente precisa percorrer toda a extensão do vetor.

5.2 Insertion Sort: Comportamento Adaptativo

O Insertion Sort demonstrou um comportamento estrutural altamente adaptativo. Quando submetido a vetores previamente ordenados, o algoritmo restringe-se a comparações locais e praticamente suprime as movimentações de memória, resultando em um tempo de processamento muito próximo ao seu limite inferior garantido de **$\Omega(n)$**.

Entretanto, no cenário de vetores completamente invertidos (ordem decrescente), a mecânica do algoritmo é severamente punida: cada elemento inserido exige um deslocamento de memória (*shift*) através de múltiplas posições contíguas. Essa operação exaustiva materializa de forma expressiva o comportamento quadrático esperado da classe **$O(n^2)$**.

5.3 Selection Sort: A Invariância Assintótica

Diferentemente das abordagens adaptativas anteriores, o Selection Sort decreta sua total escassez de variação: ele não usufrui do benefício oriundo da ordenação prévia do vetor. A cada iteração, a rotina varre impiedosamente toda a porção não ordenada remanescente para localizar o menor elemento, independentemente da organização topológica dos dados. Consequentemente, seu tempo de processamento manteve-se estritamente constante nos cenários empíricos avaliados, refletindo com primor sua complexidade assintótica exata e inflexível de **$\Theta(n^2)$**.

5.4 Quick Sort e a Mitigação do Pior Caso

A literatura estabelece que a implementação ingênua do Quick Sort possui um conhecido pior caso de complexidade **$O(n^2)$**. Isso ocorre quando o pivô é escolhido inadequadamente pelos extremos do arranjo em vetores já ordenados, o que gera ramificações e partições altamente desbalanceadas na árvore de recursão.

Para neutralizar essa anomalia estrutural, a implementação adotada neste experimento seleciona o elemento central como pivô (`mid = low + (high - low) // 2`). Essa estratégia arquitetural foi substancialmente eficaz em reduzir a probabilidade de desbalanceamento e impedir o estouro da pilha de memória. Como resultado empírico, o algoritmo absteve-se da degradação quadrática e manteve um comportamento robusto e estável, ancorado na zona de eficiência log-linear de **$\Theta(n \log n)$** em absolutamente todos os cenários avaliados.

6. Considerações sobre Arquitetura de Implementação e Overhead da Linguagem

Cumpre ressaltar que os métodos de ordenação avaliados neste ensaio empírico foram codificados e executados de forma nativa e puramente interpretada na linguagem Python. Sob essa premissa arquitetural, os tempos absolutos de processamento refletem não apenas o custo assintótico intrínseco de cada algoritmo, mas também a penalidade operacional (*overhead*) imposta pelo motor do interpretador CPython.

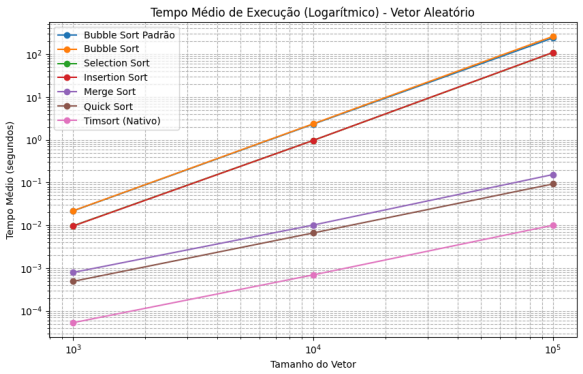
Nesse contexto, a performance registrada é impactada por três fatores sistêmicos primários:

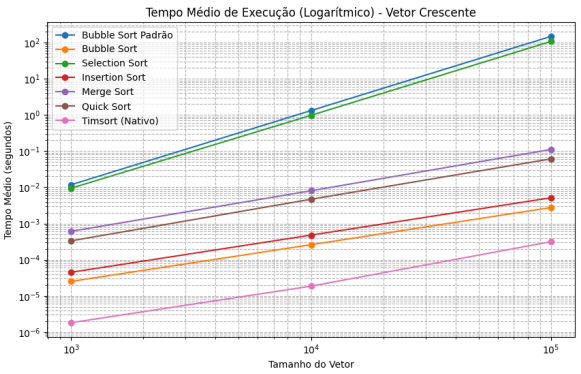
- **Tipagem Dinâmica:** A necessidade contínua de inferência, amarração e validação de tipos em tempo de execução (*runtime*) a cada operação matemática de comparação ou atribuição de variáveis.
- **Custo de Chamadas de Função (*Call Stack*):** O *overhead* gerado pelo constante empilhamento e desempilhamento de *frames* na memória, um gargalo que se torna particularmente oneroso nas arquiteturas de Divisão e Conquista (Merge Sort e Quick Sort) devido à sua profunda natureza recursiva.
- **Laços de Repetição Interpretados:** O processamento iterativo, instrução por instrução, do *bytecode* gerado pelas estruturas de controle de fluxo (*for* e *while*), que carecem das otimizações severas de compilação direta oferecidas por linguagens de baixo nível.

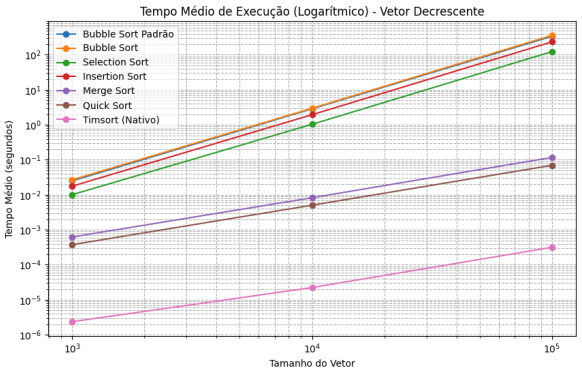
Para fins de contextualização com cenários de mercado, é pertinente destacar a função embutida de ordenação da própria linguagem (`list.sort()`). Essa função é tracionada pelo algoritmo híbrido nativo **Timsort** — uma fusão otimizada de Merge Sort e Insertion Sort. O desempenho exponencialmente superior do Timsort perante implementações customizadas não reside apenas em sua lógica adaptativa, mas principalmente no fato de ser **compilado puramente em linguagem C**. Essa característica fura a barreira do

interpretador, delegando a execução dos laços e as permutações de memória diretamente ao *hardware*.

Por conseguinte, conclui-se que, embora este ensaio evidencie e comprove categoricamente as divergências teóricas de complexidade entre as famílias de algoritmos, os valores de tempo bruto registrados ilustram os limites físicos do paradigma interpretado. Tais tempos refletem o rigor da complexidade *relativa* entre os métodos, mas não devem ser transpostos ou interpretados como o teto de performance absoluto aplicável a ambientes de produção escaláveis.







7. Conclusão

A presente avaliação experimental corrobora claramente as predições teóricas fundamentadas na análise de complexidade assintótica. Ficou empiricamente demonstrado que algoritmos de ordenação elementares — cujas arquiteturas operam na ordem quadrática de **O(n²)**, como Bubble Sort, Selection Sort e Insertion Sort — sofrem degradação drástica de escalabilidade, tornando-se computacionalmente altamente custosos perante instâncias de dados massivas. Em contrapartida, as abordagens log-lineares de **O(n log n)**, alicerçadas no paradigma de Divisão e Conquista (Quick Sort e Merge Sort), comprovaram sua robustez geométrica, mantendo tempos de execução

altamente eficientes e contidos em frações de segundo sob as mesmas condições de estresse.

Para além das métricas absolutas de tempo, a análise crítica dos ensaios evidenciou que a escolha de um método de ordenação não deve ser balizada unicamente pela sua classe Big-O. Em aplicações do mundo real, decisões de projeto de *software* exigem a ponderação rigorosa de fatores arquiteturais subjacentes, destacando-se:

- **Estabilidade:** A necessidade mandatória de preservação da ordem relativa de chaves idênticas em bases de dados complexas (propriedade garantida pelo Merge Sort).
- **Complexidade Espacial (Uso de Memória):** O compromisso entre a velocidade de processamento e a economia de alocações dinâmicas de memória por meio de operações *in-place* (vantagem inerente ao Quick Sort).
- **Sensibilidade Topológica:** A capacidade adaptativa do algoritmo perante a ordenação preliminar dos dados (ilustrada pelo contraste entre o melhor caso **Ω(n)** do Insertion Sort e a inflexibilidade estrutural do Selection Sort).

Em síntese, os resultados obtidos neste trabalho ratificam a relevância imperativa da análise formal de algoritmos não apenas como um modelo matemático abstrato, mas como a ferramenta fundamental e pragmática para a engenharia de *software* de alto desempenho. Compreender a anatomia, o comportamento temporal e as limitações sistêmicas de cada estrutura de dados é, em última instância, a premissa que orienta a construção de sistemas computacionais escaláveis, previsíveis e verdadeiramente otimizados.

8. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 4. ed. Cambridge: The MIT Press, 2015.

KARUMANCHI, N. **Data Structure and Algorithmic Thinking with Python**. New York: CareerMonk, 2015.

LEVITIN, A. V. **Introduction to the Design and Analysis of Algorithms**. New York: Addison-Wesley, 2002.

MOREIRA, Cândido. **Implementações e scripts experimentais do Projeto 1 – Algoritmos de Ordenação**. Repositório GitHub, 2026. Disponível em: https://github.com/candidomor/Mestrado-Algoritmos-Projeto1/tree/main/Principal. Acesso em: 8 mar. 2026.